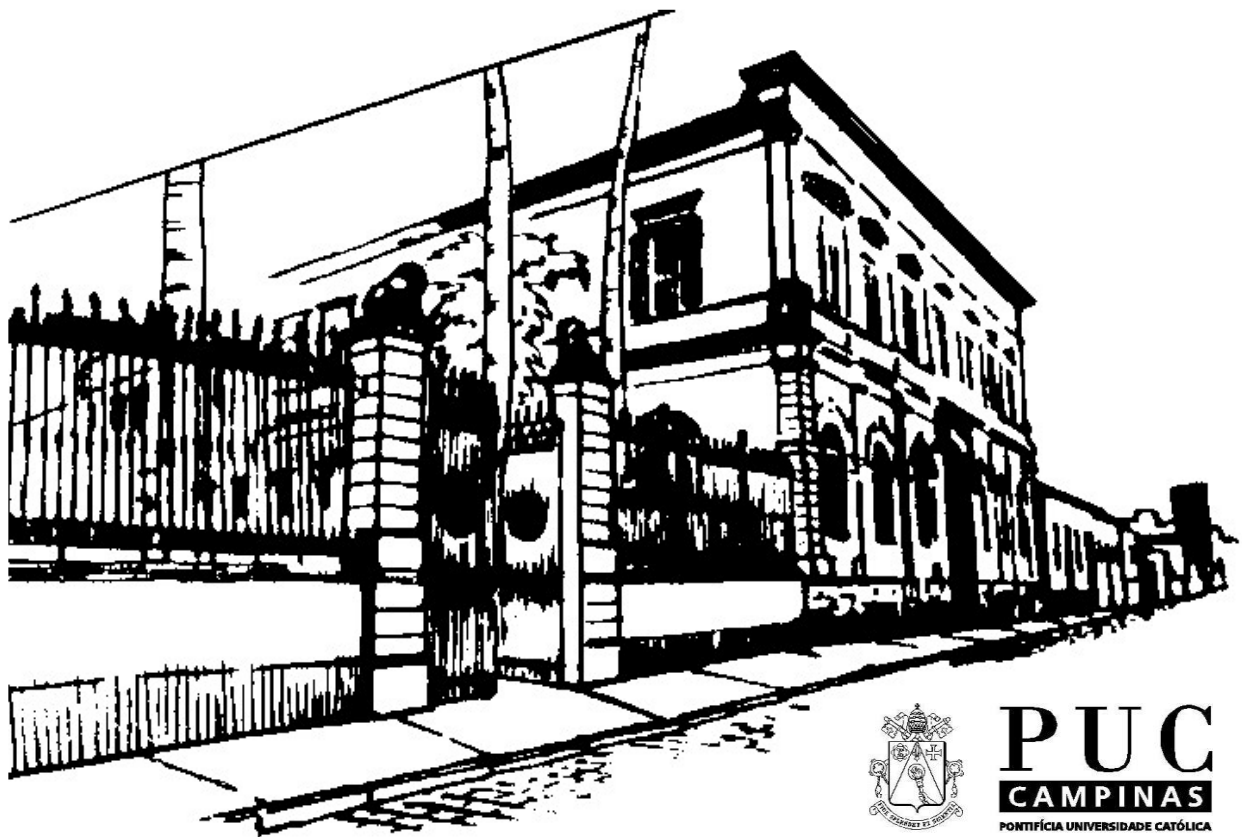




# **Escola Politécnica da PUCC**

*Faculdade de Análise de Sistemas  
Curso de Engenharia de Software*

## **Tópicos de Tecnologia e de Programação**

**1º Semestre de 2026  
Volume 4**

**Prof. André Luís dos R.G. de Carvalho**

# Índice

|                                                                   |    |
|-------------------------------------------------------------------|----|
| ÍNDICE .....                                                      | 2  |
| CAPÍTULO I: O PARADIGMA FUNCIONAL.....                            | 4  |
| INTRODUÇÃO.....                                                   | 5  |
| FUNÇÕES MATEMÁTICAS .....                                         | 6  |
| CONCLUSÃO.....                                                    | 8  |
| ANEXO I: REFERÊNCIAS .....                                        | 10 |
| CAPÍTULO II: A LINGUAGEM DE PROGRAMAÇÃO LISP .....                | 11 |
| INTRODUÇÃO.....                                                   | 12 |
| CONJUNTOS DE DADOS.....                                           | 14 |
| ÁTOMOS .....                                                      | 14 |
| S-EXPRESSÕES.....                                                 | 14 |
| LISTAS.....                                                       | 15 |
| LISP PURO .....                                                   | 15 |
| PROGRAMAÇÃO EM LISP .....                                         | 16 |
| CONSTANTES .....                                                  | 16 |
| IDENTIFICADORES DE PARÂMETROS FORMAIS.....                        | 17 |
| CHAMADA DE FUNÇÃO.....                                            | 17 |
| EXPRESSÕES CONDICIONAIS.....                                      | 18 |
| DEFINIÇÕES DE FUNÇÃO.....                                         | 19 |
| M-EXPRESSÕES .....                                                | 19 |
| CHAMADA DE FUNÇÃO.....                                            | 20 |
| EXPRESSÕES CONDICIONAIS.....                                      | 20 |
| DEFINIÇÃO DE FUNÇÕES.....                                         | 20 |
| OBSERVAÇÃO FINAL .....                                            | 21 |
| EXEMPLO.....                                                      | 21 |
| TRADUÇÃO DE M-EXPRESSÕES PARA LISP.....                           | 21 |
| Constantes.....                                                   | 21 |
| Identificadores de Parâmetros Formais .....                       | 22 |
| Chamada de Função.....                                            | 22 |
| Expressões Condicionais .....                                     | 22 |
| Definições de Função .....                                        | 23 |
| XLISP 3.04A .....                                                 | 24 |
| $\lambda$ -EXPRESSÕES.....                                        | 24 |
| PRINCIPAIS FUNÇÕES DE BIBLIOTECA .....                            | 25 |
| Parte I: Funções Aritméticas .....                                | 25 |
| Parte II: Funções de Sistema .....                                | 28 |
| Parte III: Funções de Sequência (listas, vetores e strings) ..... | 29 |
| Parte IV: Funções de Lista .....                                  | 30 |
| Parte V: Funções de Bit.....                                      | 31 |

|                                                      |           |
|------------------------------------------------------|-----------|
| <i>Parte V: Funções de String</i> .....              | 32        |
| <i>Parte VI: Predicados</i> .....                    | 32        |
| <b>ANEXO I: EXERCÍCIOS PROPOSTOS</b> .....           | <b>35</b> |
| <b>EXERCÍCIOS COM NÚMEROS</b> .....                  | <b>35</b> |
| <b>EXERCÍCIOS SOBRE LISTAS</b> .....                 | <b>41</b> |
| <b>EXERCÍCIOS SOBRE LISTAS DE NÚMEROS</b> .....      | <b>44</b> |
| <b>EXERCÍCIOS SOBRE LISTAS GENERALIZADAS</b> .....   | <b>45</b> |
| <b>EXERCÍCIOS SOBRE LISTAS DE S-EXPRESSÕES</b> ..... | <b>48</b> |
| <b>ANEXO II: REFERÊNCIAS</b> .....                   | <b>50</b> |

# **Capítulo I: O Paradigma Funcional**

## Introdução

O paradigma funcional se baseia em funções e composições de funções matemáticas, e constitui base de projeto de uma das mais importantes classes de linguagens de programação não imperativas [Sebesta89].

O objetivo de uma linguagem funcional é imitar, de forma mais perfeita possível, as funções matemáticas. Como resultado, a abordagem à solução de problemas via uma linguagem funcional é fundamentalmente diferente da abordagem via uma linguagem imperativa.

Do mesmo modo que, em linguagens imperativas, um programa é constituído pela combinação de comandos de entrada e saída, de atribuição, e de controle de fluxo de execução, em linguagens funcionais, um programa é constituído por uma série de definições de função e de definições de aplicação de função. A execução de um programa funcional consiste da avaliação das aplicações de função especificadas.

Toda linguagem funcional provê ao programador um conjunto de funções primitivas, um conjunto de formas funcionais para a construção de funções mais elaboradas, uma operação de aplicação de função, e algumas estruturas de dados. Em linguagens funcionais bem definidas existe apenas um pequeno número de funções primitivas [Sebesta89].

Em um programa funcional, funções complicadas são sempre escritas a partir de funções mais simples, e estas últimas, a partir de funções ainda mais simples, até estarem envolvidas apenas funções primitivas da linguagem [Ghezzi/Jazayeri82].

Linguagens puramente funcionais não possuem nenhuma das estruturas fundamentais de uma linguagem imperativa. Desta forma, não se encontra nelas nem variáveis, nem comandos de atribuição, nem estruturas iterativas, o que faz com que o programador deixe de ter que se preocupar com células de memória e com a forma da execução dos programas.

Embora linguagens de programação funcionais possam ser compiladas, normalmente elas são implementadas como interpretadores.

---

## Funções Matemáticas

Uma função consiste de um mapeamento de elementos de um conjunto em elementos de outro conjunto. Possuem três componentes básicos: o domínio, o contradomínio, e a definição. Podem ou não ter nome.

O domínio representa o conjunto dos objetos sobre os quais a função pode ser aplicada. Note que o domínio pode ser o produto cartesiano de diversos conjuntos.

O contradomínio representa o conjunto dos objetos que podem resultar da aplicação da função. E por fim, a definição é a especificação de como um elemento do contradomínio é determinado a partir de um elemento do domínio.

A título de ilustração, encontra-se definida abaixo a função **Quadrado**, que toma um número real e o associa a seu quadrado. A notação utilizada é a notação matemática.

$$\begin{aligned}\text{Quadrado}: & \mathbb{R} \rightarrow \mathbb{R} \\ \text{Quadrado}(x) &= x * x\end{aligned}$$

Na primeira linha desta definição, observa-se a especificação do nome, do domínio e do contradomínio da função; e na segunda linha, a especificação da associação realizada por ela.

A aplicação de uma função é especificada pela justaposição de seu nome e do elemento do domínio ao qual se quer aplicá-la.

O correspondente elemento do contradomínio é obtido pela avaliação da expressão de associação, durante a qual, instanciar-se-á todas as ocorrências de variáveis parametrizantes da expressão, de acordo com o elemento do domínio ao qual se está aplicando a função.

A título de ilustração, é mostrada abaixo a aplicação da função **Quadrado** ao número real 13. O resultado será 169.

$$\text{Quadrado}(13)$$

Não é necessário que as funções tenham nome. A notação lambda, conforme idealizada por Church em 1941 [Dershem/Jipping90], provê meios para a definição de funções sem nome. Uma expressão lambda especifica tanto os argumentos, como a expressão de associação entre

elementos do domínio a elementos do contradomínio. Veja abaixo, a forma geral da definição de uma expressão lambda .

$$\lambda\langle\text{argumentos}\rangle.\langle\text{associação}\rangle$$

Os argumentos de uma expressão lambda são, muitas vezes, chamados de variáveis livres. Antes da avaliação da expressão lambda, eles representam qualquer elemento do domínio, mas durante sua avaliação, ele é instanciado como um membro determinado.

A título de ilustração, encontra-se definida abaixo uma expressão lambda que toma um número real e o associa a seu quadrado. A notação utilizada é a notação matemática.

$$\lambda X.X * X$$

Quando uma expressão lambda é avaliada para dados argumentos, diz-se que a expressão foi aplicada àqueles argumentos.

Os mecanismos de aplicação são os mesmos para qualquer avaliação de função. Expressões lambda, como qualquer outra definição de função, podem ter mais do que um parâmetro. Veja abaixo a forma geral da aplicação de uma expressão lambda.

$$\lambda\langle\text{argumentos}\rangle.\langle\text{associação}\rangle:\langle\text{aplicação}\rangle$$

A expressão abaixo ilustra a aplicação da expressão lambda definida anteriormente que associa um número real a seu quadrado. A aplicação resultará no valor 169.

$$\lambda X.X * X : 13$$

Uma característica fundamental das funções matemáticas é que a ordem de avaliação de suas expressões de associação é controlada por expressões condicionais e por recursão, em vez de ser controlada por seqüenciação e iteração como nas linguagens de programação imperativas.

Funções matemáticas complexas são definidas em termos de outras funções. Uma função de alta ordem, ou forma funcional, é uma função que recebe funções como parâmetros e/ou retorna funções como resultado.

Uma função de alta ordem muito conhecida, é a composição de funções. Esta forma funcional toma duas funções como parâmetro e retorna uma função cujo valor é a aplicação de seu

primeiro parâmetro ao resultado de seu segundo parâmetro. Veja abaixo a definição da função  $H$ , que é a composição das funções  $F$  e  $G$ .

$$H \equiv F \circ G$$

## Conclusão

É natural comparar os paradigmas imperativo e funcional. As linguagens que seguem o paradigma imperativo se baseiam diretamente no modelo de computação de von Neumann, o que torna a programação excessivamente trabalhosa, fazendo com que os programadores tenham que se preocupar com detalhes da arquitetura da máquina que são irrelevantes no contexto do problema que pretendem resolver computacionalmente. Por outro lado, consegue-se programas que executam muito eficientemente.

Já nas linguagens que seguem o paradigma funcional, existe o que é chamado de transparência referencial de variáveis, i.e., todas as inevitáveis referências a variáveis são controladas pela linguagem de modo transparente ao programador, que pode se concentrar no problema que tem para resolver, em vez de ter que se preocupar com detalhes do modelo von Neumann de computação.

Resulta disso muito menos esforço dispendido em programação, mas também programas muito menos eficientes.

As linguagens funcionais têm uma estrutura sintática muito simples, contrastando com a estrutura sintática complexa que normalmente são encontradas nas linguagens que seguem o paradigma imperativo de programação.

É muito trabalhoso escrever programas concorrentes adotando-se uma abordagem imperativa. O programador precisa dividir estaticamente o programa em partes concorrentes, e então implementá-las como tarefas.

Conceitos complexos estão envolvidos neste trabalho, tais como, cooperação entre tarefas, competição pelo processador, acesso exclusivo a estruturas de dados compartilhadas, etc. O processo como um todo é extremamente complicado e difícil de projetar, implementar, e, principalmente, testar [Sebesta89].

---

Numa abordagem funcional, os programas podem ser reduzidos a grafos, e então ser executados pelo processo de redução de grafos, o que pode ser feito com uma boa dose de concorrência.

Isto pode ser feito dinamicamente pelo sistema de execução, o que resulta em programas altamente manuteníveis e adaptáveis à máquina em que está rodando. Isto simplifica sobremaneira a programação, torna os programas mais legíveis, e alivia o programador de uma sobrecarga desnecessária.

Além disto, embora menos eficientes do que os programas imperativos em máquina convencionais, a programação funcional pode ser muito mais eficiente em máquinas paralelas, cada vez mais comuns em nossos dias.

## Anexo I: Referências

[Sebesta89] Sebesta, R.W., “Concepts of Programming Languages”, The Benjamin/Cummings Publishing Company, Inc., 1989.

[Ghezzi/Jazayeri82] Ghezzi, C., Jazayeri, M., “Programming Languages Concepts”, John Wiley & Sons, Inc., 1982.

[Dershem/Jipping90] Dershem, H.L.; and Jipping, M.J., “Programming Languages: Structures and Models”, Wadsworth, Inc., 1990.

# **Capítulo II:**

## **A Linguagem de Programação LISP**

## Introdução

O paradigma funcional, como a maioria dos paradigmas de programação, desenvolveu-se em paralelo ao desenvolvimento de uma linguagem de programação. No seu caso, a linguagem foi LISP, uma abreviação de *List Processing*, desenvolvida em 1958, no MIT AI Project, por John McCarthy [Baranauskas93].

Desde então, muitas outras linguagens funcionais foram desenvolvidas, mas LISP é a mais antiga, e, ainda hoje, a mais usada delas.

Apesar de ter evoluído muito nestes últimos anos, muitos acham que ela não mais representa os conceitos atuais de programação funcional. Inclusive porque, excetuando-se sua primeira versão, todas as versões de LISP incluem alguma característica do paradigma imperativo, e.g., variáveis, atribuição e iteração.

Mas a despeito disto tudo isto, e de sua sintaxe estranha, LISP ainda representa bem os conceitos fundamentais da programação funcional [Sebesta89].

LISP foi originalmente projetada para atender às necessidades das primeiras aplicações em inteligência artificial, e por esta razão, guarda, ainda hoje, influências fortes do interesse dos grupos de pesquisa de então. Foi criada em uma época em que todo processamento era numérico, para atender uma enorme demanda por processamento simbólico e de listas.

Das pesquisas de McCarthy sobre computação simbólica, originou-se uma lista das características que uma linguagem para atender àqueles interesses deveria ter. Dentre elas estavam: método de controle de fluxo das funções matemáticas, i.e., recursão e expressões condicionais; e reaproveitamento automático de posições de memória não mais utilizadas.

Em LISP puro só existem dois tipos de dados: átomos e listas. Átomos são os símbolos de LISP. Constantes numéricas também são considerados átomos. Listas são coleções de elementos delimitados por parênteses. Os elementos de uma lista podem ser átomos ou então outras listas.

---

Originalmente foi definida uma notação para programas Lisp chamada Meta Expressões, ou M-expressões. Tencionava-se ter um compilador que traduzisse desta notação para código de máquina do IBM 704.

Depois, inspirado nas máquinas de Turing e motivado por certas idéias futuristas que tinha sobre processamento de listas, McCarthy teve a idéia de se ter em LISP uma função universal, que fosse capaz de avaliar qualquer outra função LISP.

O primeiro requisito para a construção desta função universal era ter uma notação única para representar dados e funções.

A notação parentizada de listas já tinha sido adotada em LISP para a representação dos dados, assim, decidiu-se criar convenções que permitissem a expressão de definições de função e de chamadas de função também sob a forma de lista.

Decidiu-se representar chamadas de função sob a forma de lista prefixa ou notação polonesa. Assim, as chamadas de função foram notadas como segue.

(NomeDaFuncao Arg<sub>1</sub> Arg<sub>2</sub> ... Arg<sub>n</sub>)

Optou-se pelo uso de expressões lambda para a especificação da definição de funções. Foi feita uma adaptação da notação matemática destas expressões para sua representação em listas.

A fim de que as expressões lambda pudessem fazer referência umas às outras, resolveu-se associar a elas um nome. Veja abaixo como são representadas as expressões lambda.

(LAMBDA (Arg<sub>1</sub> Arg<sub>2</sub> ... Arg<sub>n</sub>) Associação)

McCarthy conseguiu desenvolver sua função universal para avaliar qualquer estrutura de dados representando código LISP. A função foi batizada de **Eval**. Foi percebido então, que uma implementação de **Eval** serviria como um interpretador da linguagem, surgindo assim o primeiro interpretador LISP.

O fato dos programas terem a mesma representação que os dados pode ser explorado de formas muito interessantes. Isto, combinado ao fato da maioria das implementações de LISP tomarem a

---

forma de interpretadores, o que possibilita o acesso à função **Eval** em tempo de execução dos programas, abre caminho para que o programador escreva funções LISP que escrevem e executam código LISP.

Em concordância com as idéias originais de McCarthy sobre características de uma linguagem para processamento simbólico, LISP tem mecanismos automáticos para reaproveitamento de memória não mais usada, e tem controle do fluxo de execução que segue o modelo das funções matemáticas, i.e., baseado em recursão e expressões condicionais.

Algumas características deste primeiro interpretador LISP perpetuaram-se em quase todas as implementações posteriores da linguagem. Uma delas é o uso de controle dinâmico de escopo, que, como é sabido, tem uma série de desvantagens [Sebesta89].

Algumas implementações mais recentes da linguagem abandonam esta característica histórica e implementam controle estático de escopo, não apenas resolvendo os problemas inerentes ao controle dinâmico, mas ainda tornando os programas mais legíveis, e simplificando ligeiramente a compilação.

## Conjuntos de Dados

### Átomos

Entende-se que átomos são símbolos indivisíveis, ou seja, não se trata de uma cadeia de símbolos individualizáveis, trata-se de algo sem partes e só pode ser manipulado como um todo.

Átomos podem ser (1) inteiros, e.g., -7, 0 e 5; (2) reais, e.g., -9.7, 0.0 e 3.14; ou (3) simbólicos, e.g., pera, azul e urso.

Fica assim definido A, o conjunto dos átomos.

### S-Expressões

Seja A o conjunto dos átomos. Definimos S, o conjunto das S-expressões, da seguinte maneira:

---

- $A \supset S$ ;
- Se  $\alpha$  e  $\beta \in S$ , então o par  $(\alpha . \beta) \in S$ .

Assim, temos que  $-3$ ,  $5.9$ ,  $\text{amarelo}$ ,  $(1 . 2)$ ,  $(\text{uva} . 18.00)$ ,  $((1 . 2) . (3 . 4))$  e  $(1 . (2 . (3 . \text{nil})))$  são S-expressões.

## Listas

Seja  $S$  o conjunto das S-expressões. Definimos  $L$ , o conjunto das listas, da seguinte maneira:

- $\text{nil} \in L$ ;
- Se  $\alpha \in S$  e  $\beta \in L$ , então o par  $(\alpha . \beta) \in L$ .

Assim, temos que  $\text{nil}$ ,  $(1 . \text{nil})$ ,  $(1 . (2 . \text{nil}))$ ,  $(1 . (2 . (3 . \text{nil})))$ ,  $(1 . (2 . (3 . (4 . \text{nil}))))$  e  $(7 . (3.14 . (\text{banana} . (((1 . 2) . (3 . 4)) . ((a . (b . (c . (d . \text{nil})))) . \text{nil}))))$  são listas.

É fácil perceber que essa notação é pouco confortável, além de tornar pouco legíveis as listas escritas na mesma.

Felizmente existe uma notação alternativa para tanto. Vemos logo em seguida as mesmas listas que observamos acima, só que agora escritas nessa notação simplificada:  $()$ ,  $(1)$ ,  $(1 2)$ ,  $(1 2 3)$ ,  $(1 2 3 4)$  e  $(7 3.14 \text{ banana } ((1 . 2) . (3 . 4)) (a b c d))$ .

Deve-se ressaltar que as listas denotadas segundo essa notação simplificada são efetivamente conforme expressas na primeira notação, e, portanto quando formos manipula-las, não devemos perder esse fato de vista.

## LISP Puro

LISP puro tem 5 funções primitivas ( $\text{car}$ ,  $\text{cdr}$ ,  $\text{cons}$ ,  $1+$  e  $1-$ ), 2 predicados ( $\text{equal}$  e  $\text{atom}$ ) e 2 formas ( $\text{cond}$  e  $\text{lambda}$ ).

- $\text{Car}: S-A \rightarrow S$

$\text{Car}$  aplicado a um par pontuado  $(\alpha . \beta)$  resulta  $\alpha$ , ou seja, resulta o primeiro elemento de um par pontuado.

---

- **Cdr:**  $S-A \rightarrow S$   
Cdr aplicado a um par pontuado  $(\alpha . \beta)$  resulta  $\beta$ , ou seja, resulta o segundo elemento de um par pontuado.
- **Cons:**  $S \times S \rightarrow S-A$   
Cons aplicado a duas S-Expressões  $\alpha$  e  $\beta$ , resulta o par pontuado  $(\alpha . \beta)$ , ou seja, resulta um par pontuado constituído por  $\alpha$  e  $\beta$ .
- **Equal:**  $A \times A \rightarrow \{t, nil\}$   
Equal aplicado a dois Átomos  $\alpha$  e  $\beta$ , resulta o átomo 't (verdadeiro), caso  $\alpha$  e  $\beta$  sejam átomos idênticos, ou 'nil (falso), caso contrário.
- **Atom:**  $S \rightarrow \{t, nil\}$   
Atom aplicado a uma S-Expressão  $\alpha$ , resulta o átomo 't (verdadeiro), caso  $\alpha$  seja um átomo, ou 'nil (falso), caso contrário (caso  $\alpha$  seja um par pontuado).
- **1+:**  $R \rightarrow R$   
1+ aplicado a um número real  $\alpha$ , resulta  $\alpha+1$ .
- **1-:**  $R \rightarrow R$   
1- aplicado a um número real  $\alpha$ , resulta  $\alpha-1$ .

## Programação em LISP

A escrita de um programa em LISP ocorre de forma simples, já que nesta linguagem não existe uma grande variedade de formas sintáticas. Abaixo encontraremos as regras para escrever programas em LISP:

### Constantes

Constantes podem ser escritas de duas formas: (1) simplesmente acrescentando uma apóstrofe (') no início da constante; ou (2) envolvendo a constante entre parênteses e a precedendo da palavra reservada quote.

---

Constantes que são átomos numéricos, embora possam ser escritas como explicado acima, podem, alternativamente, serem escritas de forma bem mais simples, ou seja, simplesmente o valor da constante, sem nenhuma notação especial.

Em algumas implementações de Lisp, a mesma simplificação acima explicada para átomos numéricos, se aplica também aos outros tipos de átomos, ou seja, aos átomos simbólicos.

Veja os exemplos abaixo:

| Constante        | Lisp                                          |
|------------------|-----------------------------------------------|
| 1                | '1 ou (quote 1) ou ainda 1                    |
| 3.14             | '3.14 (quote 3.14) ou ainda 3.14              |
| Abacaxi          | 'abacaxi ou (quote abacaxi) ou ainda abacaxi  |
| (abacate . 5.00) | '(abacate . 5.00) ou (quote (abacate . 5.00)) |
| (1 2 3 4 5 6 7)  | '(1 2 3 4 5 6 7) ou (quote (1 2 3 4 5 6 7))   |

## Identificadores de Parâmetros Formais

Identificadores de parâmetros formais são escritos de forma muito simples; são simplesmente escritos, sem nenhuma notação especial.

Veja os exemplos abaixo:

| Parâmetro Formal | Lisp |
|------------------|------|
| A                | A    |
| X                | X    |

## Chamada de Função

A escrita de uma chamada de função também não se acontece de forma complicada. Basta mencionar, entre parênteses, o nome da função, seguidos seguir o que indica a forma geral de tradução indicada abaixo:

| Nome da Função | Parâmetros Reais<br>(apropriadamente escritos, conforme sua natureza) | Chamada em Lisp                                          |
|----------------|-----------------------------------------------------------------------|----------------------------------------------------------|
| F              | PR <sub>1</sub> , PR <sub>2</sub> , ... , e PR <sub>n</sub>           | (F PR <sub>1</sub> PR <sub>2</sub> ... PR <sub>n</sub> ) |

Veja os exemplos abaixo:

| <b>Nome da Função</b> | <b>Seus Parâmetros Reais</b><br>(apropriadamente escritos, conforme sua natureza) | <b>Chamada em Lisp</b>       |
|-----------------------|-----------------------------------------------------------------------------------|------------------------------|
| Car                   | '((a . b) . (c . d))                                                              | (Car '((a . b) . (c . d)))   |
| Cdr                   | '(a . (b . (c . nil)))                                                            | (Cdr '(a . (b . (c . nil)))) |
| Equal                 | X e 3                                                                             | (Equal X 3)                  |

Onde X deve ser entendido como sendo um parâmetro formal de uma função dentro da qual a chamada da função Equal estaria ocorrendo.

## Expressões Condicionais

A escrita de Expressões Condicionais também não se dá de forma complicada. Basta seguir o que indica a forma geral abaixo:

| <b>Expressões Lógicas</b><br>(apropriadamente escritas, conforme sua natureza) | <b>Possíveis Resultados</b><br>(apropriadamente escritos, conforme sua natureza) | <b>Lisp</b>                                                                                                      |
|--------------------------------------------------------------------------------|----------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|
| $EL_1, EL_2, \dots, EL_n$                                                      | $R_1, R_2, \dots, R_n$                                                           | (Cond (EL <sub>1</sub> R <sub>1</sub> ) (EL <sub>2</sub> R <sub>2</sub> ) ... (EL <sub>n</sub> R <sub>n</sub> )) |

onde (1) os  $EL_i$  representam expressões que resultam S-expressão  $\in \{t, nil\}$ , condições para a produção de cada um dos  $R_i$ ; (2) os  $R_i$  representam expressões que resultam S-expressões quaisquer, resultados da expressão condicional no caso da  $EL_i$  que lhe corresponde valer t.

Veja o exemplo abaixo:

| <b>Expressões Lógicas</b><br>(apropriadamente escritas, conforme sua natureza) | <b>Possíveis Resultados</b><br>(apropriadamente escritos, conforme sua natureza) | <b>Lisp</b>                                                                  |
|--------------------------------------------------------------------------------|----------------------------------------------------------------------------------|------------------------------------------------------------------------------|
| (Atom L)                                                                       | 'nil                                                                             | (Cond ((Atom L) 'nil)<br>((Equal A (Car L)) 't)<br>( 't (Ocorre A (Cdr L)))) |
| (Equal A (Car L))                                                              | 't                                                                               |                                                                              |
| 't                                                                             | ( 't (Ocorre A (Cdr L)))                                                         |                                                                              |

Onde A e L devem ser entendidos como sendo parâmetros formais de uma função cuja definição seria dada pela expressão condicional em questão e Ocorre deve ser entendida como sendo o

nome de uma função que estaria sendo chamada na expressão condicional, exatamente da mesma forma que acontece com as funções Atom, Equal, Car e Cdr.

## Definições de Função

Definições de Função são escritas de forma muito simples. Basta seguir o que indica a forma geral abaixo:

| Nome da Função | Seu domínio | Seu contra-domínio | Seus Parâmetros Formais<br>(apropriadamente escritos, conforme sua natureza) | Sua Expressão de Definição<br>(apropriadamente escrita, conforme sua natureza) | Lisp                                                          |
|----------------|-------------|--------------------|------------------------------------------------------------------------------|--------------------------------------------------------------------------------|---------------------------------------------------------------|
| F              | D           | C                  | $P_1, P_2, \dots, P_n$                                                       | E                                                                              | $; F: D \rightarrow C$<br>(defun F ( $P_1 P_2 \dots P_n$ ) E) |

onde (1) F representa o nome da função a ser definida; (2) D representa seu domínio; (3) C representa seu contradomínio; (4)  $P_i$  representam seus parâmetros formais; e (5) E representa sua expressão de definição (em geral é uma chamada de função ou uma expressão condicional).

Veja o exemplo abaixo, que apresenta uma função que verifica se um dado átomo ocorre em uma dada lista de átomos, resultando o átomo 't (verdadeiro), em caso afirmativo, ou o átomo 'nil (falso), caso contrário:

```
; Contem: LxA → {'t, 'nil}
(Defun Contem (L A)
  (Cond ((Atom L) 'nil)
        ((Equal A (Car L)) 't)
        ('t (Ocorre A (Cdr L)))))
```

## M-Expressões

Como sabemos, a sintaxe que tem a linguagem LISP não é aquela que originalmente foi concebida para ela. LISP tem uma sintaxe pouco confortável, com muitos parênteses, muito diferente daquela que a princípio se concebia para a linguagem.

M-expressões representam uma sintaxe bem mais simples e muito mais fácil de ser empregada. Por esta razão, M-expressões são freqüentemente usadas como uma linguagem algorítmica para a escrita de algoritmos funcionais.

Tais algoritmos, posteriormente, poderão ser traduzidos para uma linguagem funcional real, LISP, por exemplo.

## ***Chamada de Função***

A forma geral de uma chamada de função é a seguinte:

$$F [PR_1; PR_2; \dots ; PR_n]$$

onde (1) F é o nome da função; e (2) os  $PR_i$ 's representam os parâmetros reais da função.

## ***Expressões Condicionais***

Expressões condicionais resultam em S-expressões. Sua forma geral é a seguinte:

$$[EL_1 \rightarrow R_1; EL_2 \rightarrow R_2; \dots ; EL_n \rightarrow R_n]$$

onde (1) os  $EL_i$  representam expressões que resultam S-expressão que  $\in \{t, nil\}$ ; e (2) os  $R_i$  representam expressões que resultam S-expressões quaisquer.

Expressões condicionais são avaliadas da seguinte forma: (1) avalia-se cada uma das  $EL_i$  da esquerda para a direita; (2) ao se encontrar uma cujo valor seja t fica determinado que o valor da expressão condicional é  $R_i$ ; (3) se todas as  $EL_i$  resultarem nil, o valor da expressão condicional é indefinido.

## ***Definição de Funções***

A forma geral de uma definição de função é a seguinte:

$$\begin{aligned} F: D &\rightarrow C \\ F [PF_1; PF_2; \dots ; PF_n] &= E \end{aligned}$$

onde (1)  $F$  é o nome da função; (2)  $D$  representa o domínio da função  $F$ ; (3)  $C$  representa o contradomínio da função  $F$ ; (4) os  $PF_i$ 's representam os parâmetros formais da função  $F$ ; e (5)  $E$  é a expressão de associação entre elementos de  $D$  e elementos de  $C$  e representa uma expressão que resulta uma S-expressão.

Vale ressaltar que  $E$  pode ser uma constante, uma chamada de função ou uma expressão condicional.

## **Observação Final**

Por convenção, empregaremos somente letras minúsculas para escrever átomos simbólicos, ao passo que as palavras que compõem os nomes de identificadores iniciar-se-ão sempre com uma letra maiúscula.

## **Exemplo**

Abaixo encontraremos a definição de um predicado que verifica se um dado átomo ocorre em uma lista de átomos.

Ocorre:  $AxL \rightarrow \{t, nil\}$

Ocorre  $[A;L] = [Atom [L] \rightarrow nil; Equal [A; Car [L]] \rightarrow t; t \rightarrow Ocorre [A; Cdr [L]]]$

## **Tradução de M-Expressões para LISP**

A tradução de um programa escrito sob a forma de M-expressões para LISP ocorre de forma simples e mecânica. Abaixo encontraremos as regras para realizar a referida tradução:

### **Constantes**

Constantes são traduzidas de forma muito simples. Átomos numéricos não sofrem nenhuma transformação no processo de tradução.

As demais constantes podem ser traduzidas de duas formas: (1) simplesmente acrescentando uma apóstrofe (') no início da constante; ou (2) envolvendo a constante entre parênteses e a precedendo da palavra reservada quote.

Veja os exemplos abaixo:

---

| <b>M-Expressão</b> | <b>Lisp</b>                                   |
|--------------------|-----------------------------------------------|
| 1                  | 1                                             |
| 3.14               | 3.14                                          |
| abacaxi            | 'abacaxi ou (quote abacaxi)                   |
| (abacate . 5.00)   | '(abacate . 5.00) ou (quote (abacate . 5.00)) |
| (1 2 3 4 5 6 7)    | '(1 2 3 4 5 6 7) ou (quote (1 2 3 4 5 6 7))   |

### Identificadores de Parâmetros Formais

Identificadores de parâmetros formais são traduzidas de forma muito simples; não sofrem nenhuma transformação no processo de tradução.

Veja os exemplos abaixo:

| <b>M-Expressão</b> | <b>Lisp</b> |
|--------------------|-------------|
| A                  | A           |
| X                  | X           |

### Chamada de Função

A tradução de Expressões Condicionais também não se acontece de forma complicada. Basta seguir o que indica a forma geral de tradução indicada abaixo:

| <b>M-Expressão</b>                                             | <b>Lisp</b>                                                 |
|----------------------------------------------------------------|-------------------------------------------------------------|
| F [PR <sub>1</sub> ; PR <sub>2</sub> ; ... ; PR <sub>n</sub> ] | (F TPR <sub>1</sub> TPR <sub>2</sub> ... TPR <sub>n</sub> ) |

onde (1) F é o nome da função; (2) os PR<sub>i</sub>'s representam os parâmetros reais da função; e (3) os TPR<sub>i</sub>'s representam a tradução dos PR<sub>i</sub>'s.

Veja os exemplos abaixo:

| <b>M-Expressão</b>   | <b>Lisp</b>          |
|----------------------|----------------------|
| Equal [A; nil]       | (Equal A 'nil)       |
| Equal [Cdr [A]; nil] | (Equal (Cdr A) 'nil) |

### Expressões Condicionais

A tradução de Expressões Condicionais também não se dá de forma complicada. Basta seguir o que indica a forma geral de tradução abaixo:

| <b>M-Expressão</b>                                                                                              | <b>Lisp</b>                                                                                                            |
|-----------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| [EL <sub>1</sub> → R <sub>1</sub> ; EL <sub>2</sub> → R <sub>2</sub> ; ... ; EL <sub>n</sub> → R <sub>n</sub> ] | (Cond (TEL <sub>1</sub> TR <sub>1</sub> ) (TEL <sub>2</sub> TR <sub>2</sub> ) ... (TEL <sub>n</sub> TR <sub>n</sub> )) |

onde (1) os  $EL_i$  representam expressões que resultam S-expressão que  $\in \{t, nil\}$ ; (2) os  $R_i$  representam expressões que resultam S-expressões quaisquer; (3) os  $TEL_i$  representam a tradução dos  $EL_i$ ; e (4) os  $TR_i$  representam a tradução dos  $R_i$ .

Veja o exemplo abaixo:

| M-Expressão                                                                                                | Lisp                                                                              |
|------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| $[Atom [L] \rightarrow nil;$<br>$Equal [A; Car [L]] \rightarrow t;$<br>$t \rightarrow Ocorre [A; Cdr [L]]$ | $(Cond ((Atom L) 'nil)$<br>$((Equal A (Car L)) 't)$<br>$('t (Ocorre A (Cdr L))))$ |

### Definições de Função

A tradução de Definições de Função também é simples. Basta seguir o que indica a forma geral de tradução abaixo:

| M-Expressão                                               | Lisp                                                               |
|-----------------------------------------------------------|--------------------------------------------------------------------|
| $F: D \rightarrow C$<br>$F [PF_1; PF_2; \dots; PF_n] = E$ | $; F: D \rightarrow C$<br>$(Defun F (TPF_1 TPF_2 \dots TPF_n) TE)$ |

onde (1)  $F$  é o nome da função; (2)  $D$  representa o domínio da função  $F$ ; (3)  $C$  representa o contradomínio da função  $F$ ; (4) os  $PF_i$ 's representam os parâmetros formais da função  $F$ ; (5)  $E$  é a expressão de associação entre elementos de  $D$  e elementos de  $C$  e representa uma expressão que resulta uma S-expressão; (6) os  $TPF_i$ 's representam a tradução dos  $PF_i$ 's; e (7)  $TE$  representa a tradução de  $E$ .

Veja o exemplo abaixo:

| M-Expressão                                                                                                                                                        | Lisp                                                                                                                                                 |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| $Ocorre: AxL \rightarrow \{t, nil\}$<br>$Ocorre [A; L] = [Atom [L] \rightarrow nil;$<br>$Equal [A; Car [L]] \rightarrow t;$<br>$t \rightarrow Ocorre [A; Cdr [L]]$ | $; Ocorre: AxL \rightarrow \{t, nil\}$<br>$(Defun Ocorre (A L)$<br>$(Cond ((Atom L) 'nil)$<br>$((Equal A (Car L)) 't)$<br>$('t (Ocorre A (Cdr L))))$ |

## XLISP 3.04A

### $\lambda$ -Expressões

Como já discutimos anteriormente, em LISP, programa e dados têm a mesma forma, podendo (1) o programa ser visto e manipulado como uma estrutura de dados; e (2) as estruturas de dados serem vistas e executadas como um trecho de programa.

$\lambda$ -expressões são o que empregamos para representarmos funções como estruturas de dados manipuláveis.

Vimos na primeira parte desta apostila o que significam conceitualmente as  $\lambda$ -expressões, veremos a seguir de que forma uma função é representada como uma  $\lambda$ -expressão.

Consideremos a função Ocorre conforme definida abaixo:

```
(Defun Ocorre (A L)
  (Cond ((Atom L) 'nil)
        ((Equal A (Car L)) 't)
        ('t (Ocorre A (Cdr L)))))
```

Quando o interpretador LISP tratar essa definição, este promoverá a definição de um símbolo de nome Ocorre que terá como valor a seguinte lista:

```
'(Lambda (A L)
  (Cond ((Atom L) nil)
        ((Equal A (Car L)) t)
        (t (Ocorre A (Cdr L)))))
```

Podemos entender o símbolo Ocorre como se fosse uma variável que contém a lista acima. Para, a partir dele, obter a lista que ele representa, basta aplicar as seguintes funções como segue:

```
(get-lambda-expression (function Ocorre))
```

Desta forma, poderemos manipular a referida lista, por exemplo, tomando dela uma parte. Assim, se calcularmos,

```
(Car (Cdr (Cdr (Get-Lambda-Expression (Function Ocorre)))))
```

obteremos o valor

'Cond

E a criação de símbolos não está restrita ao interpretador LISP. O programador também pode definir símbolos. Isto pode ser feito através da função **SetFun**. Veja abaixo outra alternativa para a definição da função **Ocorre**, além, naturalmente, da forma já estudada (**Defun**).

```
(SetFun 'Ocorre '(Lambda (A L)
                    (Cond ((Atom L) nil)
                          ((Equal A (Car L)) t)
                          (t (Ocorre A (Cdr L)))))
```

Note que a mesma lista poderia ser obtida não pela escrita explícita desta constante, mas pela execução de funções que a produzissem.

## ***Principais Funções de Biblioteca***

### **Parte I: Funções Aritméticas**

1. (**truncate FLT**): produz a parte inteira do real FLT;
  2. (**round FLT**): produz o arredondamento do real FLT para o inteiro mais próximo;
  3. (**floor FLT**): produz o arredondamento para baixo do real FLT;
  4. (**ceiling FLT**): produz o arredondamento para cima do real FLT;
  5. (**float INT**): produz o equivalente real do número inteiro INT;
  6. (**rational FLT**): produz o equivalente racional do número real FLT;
  7. (**+ NUM<sub>1</sub> NUM<sub>2</sub> ... NUM<sub>n</sub>**): produz a soma de todos os números que recebe como argumento;
  8. (**- NUM<sub>1</sub> NUM<sub>2</sub> ... NUM<sub>n</sub>**): recebe uma série de números como argumento; produz o primeiro deles com sinal invertido (no caso dele ser seu único argumento) ou produz a subtração de todos os demais do primeiro (no caso de receber mais de um);
-

9. (`* NUM1 NUM2 ... NUMn`): produz a multiplicação de todos os números que recebe como argumento;
  10. (`/ NUM1 NUM2 ... NUMn`): recebe uma série de números como argumento; produz o inverso de seu primeiro deles (no caso dele ser o único) ou produz a divisão do primeiro deles por todos os demais (no caso de receber mais de um);
  11. (`1+ NUM`): produz o incremento de NUM;
  12. (`1- NUM`): produz o decremento de NUM;
  13. (`rem NUM1 NUM2`) ou (`mod NUM1 NUM2`): produz o resto da divisão inteira de NUM<sub>1</sub> por NUM<sub>2</sub>;
  14. (`min NUM1 NUM2 ... NUMn`): produz o menor de todos os números que recebe como argumento;
  15. (`max NUM1 NUM2 ... NUMn`): produz o maior de todos os números que recebe como argumento;
  16. (`abs NUM`): produz o valor absoluto de NUM;
  17. (`gcd INT1 INT2 ... INTn`): produz o máximo divisor comum dos números inteiros que recebe como argumento;
  18. (`lcm INT1 INT2 ... INTn`): produz o mínimo múltiplo comum dos números inteiros que recebe como argumento;
  19. (`random FLT`): produz um número aleatório entre 0 e o número real FLT;
  20. (`setq *random-state* (make-a-random-state t)`): ajusta uma nova semente geradora de números aleatórios;  
  
(`make-a-random-state`): produz a atual semente geradora de números aleatórios;
  21. (`sin NUM`): produz o seno do valor angular NUM expresso em radianos;
  22. (`cos NUM`): produz o cosseno do valor angular NUM expresso em radianos;
-

23. (tan NUM): produz a tangente do valor angular NUM expresso em radianos;
  24. (asin NUM): produz um valor angular em radianos cujo seno vale NUM;
  25. (acos NUM): produz um valor angular em radianos cujo coseno vale NUM;
  26. (atan NUM): produz um valor angular em radianos cuja tangente vale NUM;
  27. (expt NUM<sub>1</sub> NUM<sub>2</sub>): produz NUM<sub>1</sub> elevado a NUM<sub>2</sub>;
  28. (exp NUM): produz e elevado NUM (onde e é o número de Neper);
  29. log NUM<sub>1</sub> NUM<sub>2</sub>): produz o logaritmo de NUM<sub>1</sub> na base NUM<sub>2</sub> (se NUM<sub>2</sub> for omitido, assume-se logaritmo natural);
  30. (sqrt NUM): produz a raiz quadrada de NUM;
  31. (numerator RAC): produz o numerador do número racional RAC;
  32. (denominator RAC): produz o denominador do número racional RAC;
  33. (complex FLT<sub>1</sub> FLT<sub>2</sub>): produz um número complexo com o coeficiente real FLT<sub>1</sub> e o coeficiente imaginário FLT<sub>2</sub>;
  34. (realpart CPX): produz o coeficiente real do número complexo CPX;
  35. (imagpart CPX): produz o coeficiente imaginário do número complexo CPX;
  36. (conjugate CPX): produz o conjugado do número complexo CPX;
  37. (phase CPX): produz a fase do número conjugado CPX (equivale a ***(atan (/ (imagpart CPX) (realpart CPX))))***);
  38. (= NUM<sub>1</sub> NUM<sub>2</sub> ... NUM<sub>n</sub>): produz t, caso todos os números que recebe como argumento sejam iguais; produz nil, caso contrário;
  39. (/= NUM<sub>1</sub> NUM<sub>2</sub> ... NUM<sub>n</sub>): produz t, caso seu primeiro argumento seja diferente de todos os demais; produz nil, caso contrário;
-

40. (`< NUM1 NUM2 ... NUMn`): produz `t`, caso seu primeiro argumento seja menor que todos os demais; produz `nil`, caso contrário;
41. (`<= NUM1 NUM2 ... NUMn`): produz `t`, caso seu primeiro argumento seja menor ou igual a todos os demais; produz `nil`, caso contrário;
42. (`> NUM1 NUM2 ... NUMn`): produz `t`, caso seu primeiro argumento seja maior que todos os demais; produz `nil`, caso contrário;
43. (`>= NUM1 NUM2 ... NUMn`): produz `t`, caso seu primeiro argumento seja maior ou igual a todos os demais; produz `nil`, caso contrário;

## Parte II: Funções de Sistema

1. (`setfun SMB LBD`): define a função cujo nome é dado pelo símbolo `SMB` como sendo expressa pela  $\lambda$ -expressão `LBD`;
  2. (`load STR`): carrega o arquivo de código fonte cujo nome é expresso pelo string `STR` (diretórios separados por `\` e não por `;`; assume extensão `.LSP`);
  3. (`restore STR`): carrega a área de trabalho anteriormente salva no arquivo cujo nome é expresso pelo string `STR` (diretórios separados por `\` e não por `;`; assume extensão `.WKS`);
  4. (`save STR`): salva a área de trabalho no arquivo cujo nome é expresso pelo string `STR` (diretórios separados por `\` e não por `;`; assume extensão `.WKS`);
  5. (`dribble STR`): salva no arquivo cujo nome é expresso pelo string `STR` a transcrição de uma sessão `XLisp`;
  6. (`gc`): força a coleta de lixo (que normalmente acontece de forma automática sob o controle do ambiente `XLisp`);
  7. (`expand INT`): expande a memória pelo acréscimo de `INT` novos segmentos de memória de tamanho fixo; retorna a quantidade de segmentos acrescentada;
  8. (`room`): exibe estatísticas de alocação de memória;
-

9. (time FCN): executa a função FCN e produz seu tempo de execução;
10. (/ (get-internal-real-time) internal-time-units-per-second): produz o tempo de relógio decorrido;
11. (/ (get-internal-run-time) internal-time-units-per-second): produz o tempo de execução decorrido;
12. (type-of EXP): produz o tipo da expressão EXP (LIST, se nil; CONS se lista ou par com ponto; SYMBOL, se símbolo; OBJECT, se objeto; SUBR, se função de biblioteca; FSUBR, se função de biblioteca escrita em C; CLOSURE, se função de usuário; STRING, se string; FIXNUM, se inteiro; BIGNUM, se inteiro longo; RATIO, se racional; FLONUM, se real; COMPLEX, se complexo; CHARACTER, se caractere; FILE-STREAM, se arquivo; UNNAMED-STREAM, se arquivo sem nome; ARRAY, se vetor; HASH-TABLE, se tabela de espalhamento; SYM, para estruturas “sym”);
13. (coerce EXP TIP): produz a conversão da expressão EXP para o tipo TIP (TIP pode ser qualquer um dos tipos produzidos pela função type-of);
14. (peek INT): produz o conteúdo do endereço de memória expresso pelo número inteiro INT;
15. (poke INT<sub>1</sub> INT<sub>2</sub>): coloca no endereço de memória expresso pelo número inteiro INT<sub>1</sub>, o valor expresso pelo número inteiro INT<sub>2</sub>;
16. (get-key): produz um inteiro que representa o código ASCII da tecla que for pressionada;
17. (exit): sai do XLisp;
18. (eval FCN): executa a função FCN;

### Parte III: Funções de Seqüência (listas, vetores e strings)

1. (concatenate TIP EXP<sub>1</sub> EXP<sub>2</sub> ... EXP<sub>n</sub>): produz um resultado do tipo TIP que equivale a concatenação das expressões EXP<sub>i</sub> recebidas como argumento; TIP pode ser CONS, LIST, ARRAY, ou STRING;
  2. (elt SEQ INT): produz o elemento de ordem INT da seqüência SEQ;
-

3. (length SEQ): produz o tamanho da seqüência SEQ;
4. (reverse SEQ): produz o inverso da seqüência SEQ;
5. (subseq SEQ NUM<sub>1</sub> NUM<sub>2</sub>): produz a subseqüência da seqüência SEQ que se inicia em na posição expressa pelo inteiro NUM<sub>1</sub> e que termina na posição expressa pelo inteiro NUM<sub>2</sub>;

#### Parte IV: Funções de Lista

1. (car LST): produz o primeiro elemento da lista LST;
  2. (cdr LST): produz o que resta da lista LST, retirado seu primeiro elemento;
  3. (cxxxr LST): equivale a (cxr (cxr (LST))), onde 'x' é 'a' ou 'd';
  4. (cxxxxr LST): equivale a (cxr (cxr (cxr (LST)))), onde 'x' é 'a' ou 'd';
  5. (cxxxxxr LST): equivale a (cxr (cxr (cxr (cxr (LST))))), onde 'x' é 'a' ou 'd';
  6. (first LST): equivale a (car LST);
  7. (second LST): equivale a (cadr LST);
  8. (third LST): equivale a (caddr LST);
  9. (fourth LST): equivale a (cadddr LST);
  10. (rest LST): equivale a (cdr LST);
  11. (last LST): produz o último elemento da lista LST;
  12. (cons EXP<sub>1</sub> EXP<sub>2</sub>): constroi o par com ponto (EXP<sub>1</sub> . EXP<sub>2</sub>) a partir das expressões EXP<sub>i</sub> dadas;
  13. (list EXP<sub>1</sub> EXP<sub>2</sub> ... EXP<sub>n</sub>): produz uma lista que tem por elementos as expressões EXP<sub>i</sub> dadas;
  14. (append LST<sub>1</sub> LST<sub>2</sub> ... LST<sub>n</sub>): produz uma lista que expressa a concatenação das listas LST<sub>i</sub> dadas;
  15. (list-length LST): produz um inteiro que expressa o tamanho da lista LST;
-

16. (butlast LST INT): produz uma lista que tem todos os elementos presentes na lista LST exceto os INT últimos;
17. (nth INT LST): produz o elemento de ordem INT da lista LST;
18. (nthcdr INT LST): produz o que resta da lista LST, após tomado INT vezes o seu cdr;

### Parte V: Funções de Bit

1. (logand INT<sub>1</sub> INT<sub>2</sub> ... INT<sub>n</sub>): produz o **and** bit a bit dos inteiros INT<sub>i</sub> recebidos como argumento;
  2. (logior INT<sub>1</sub> INT<sub>2</sub> ... INT<sub>n</sub>): produz o **or** bit a bit dos inteiros INT<sub>i</sub> recebidos como argumento;
  3. (logxor INT<sub>1</sub> INT<sub>2</sub> ... INT<sub>n</sub>): produz o **xor** bit a bit dos inteiros INT<sub>i</sub> recebidos como argumento;
  4. (logeqv INT<sub>1</sub> INT<sub>2</sub> ... INT<sub>n</sub>): produz a **equivalência** bit a bit dos inteiros INT<sub>i</sub> recebidos como argumento;
  5. (lognand INT<sub>1</sub> INT<sub>2</sub> ... INT<sub>n</sub>): equivale a (lognot (logand INT<sub>1</sub> INT<sub>2</sub> ...INT<sub>n</sub>));
  6. (logandc1 INT<sub>1</sub> INT<sub>2</sub>): equivale a (logand (lognot INT<sub>1</sub>) INT<sub>2</sub>);
  7. (logandc2 INT<sub>1</sub> INT<sub>2</sub>): equivale a (logand INT<sub>1</sub> (lognot INT<sub>2</sub>));
  8. (lognor INT<sub>1</sub> INT<sub>2</sub> ... INT<sub>n</sub>): equivale a (lognot (logior INT<sub>1</sub> INT<sub>2</sub> ...INT<sub>n</sub>));
  9. (logorc1 INT<sub>1</sub> INT<sub>2</sub>): equivale a (logior (lognot INT<sub>1</sub>) INT<sub>2</sub>);
  10. (logorc2 INT<sub>1</sub> INT<sub>2</sub>): equivale a (logior INT<sub>1</sub> (lognot INT<sub>2</sub>));
  11. (lognot INT): produz o **not** bit a bit do inteiro INT recebido como argumento;
  12. (logtest INT<sub>1</sub> INT<sub>2</sub>): produz t, caso o **and** bit a bit dos inteiros INT<sub>i</sub> dados produzir um valor não nulo; produz nil, caso contrário;
  13. (logbitp INT<sub>1</sub> INT<sub>2</sub>): produz t, caso o bit na posição INT<sub>1</sub> do inteiro INT<sub>2</sub> valer 1; produz nil, caso contrário;
-

14. (logcount INT): se o número inteiro INT for positivo, produz a contagem quantos bits 1 o constituem; caso contrário, produz a contagem quantos zeros constituem seu correspondente positivo;
15. (integer-length INT): produz a quantidade de bits que são necessários para representar o número inteiro INT;
16. (ash INT<sub>1</sub> INT<sub>2</sub>): desloca INT<sub>2</sub> vezes os bits do inteiro INT<sub>1</sub> (para a esquerda, se INT<sub>2</sub> for positivo; para a direita, caso contrário);

### Parte V: Funções de String

1. (string INT): produz um string contendo tão somente o caractere cujo código ASCII é expresso pelo inteiro INT;
2. (string-trim STR<sub>1</sub> STR<sub>2</sub>): produz um string em tudo idêntico a STR<sub>2</sub>, exceto pelo fato de que remove de seu início e de seu final os caracteres que constituem o string STR<sub>1</sub>;
3. (string-left-trim STR<sub>1</sub> STR<sub>2</sub>): produz um string em tudo idêntico a STR<sub>2</sub>, exceto pelo fato de que remove de seu início os caracteres que constituem o string STR<sub>1</sub>;
4. (string-right-trim STR<sub>1</sub> STR<sub>2</sub>): produz um string em tudo idêntico a STR<sub>2</sub>, exceto pelo fato de que remove de seu final os caracteres que constituem o string STR<sub>1</sub>;
5. (strcat STR<sub>1</sub> STR<sub>2</sub> ... STR<sub>n</sub>): produz a concatenação dos strings STR<sub>i</sub> fornecidos como argumento;

### Parte VI: Predicados

1. (atom EXP): produz t, caso a expressão EXP seja um átomo; produz nil, caso contrário;
  2. (symbolp EXP): produz t, caso a expressão EXP seja um símbolo; produz nil, caso contrário;
  3. (numberp EXP): produz t, caso a expressão EXP seja um número; produz nil, caso contrário;
  4. (null EXP): produz t, caso a expressão EXP seja nil; produz nil, caso contrário;
  5. (not EXP): produz t, caso a expressão EXP seja nil; produz nil, caso contrário;
-

6. (listp EXP): produz t, caso a expressão EXP seja uma lista; produz nil, caso contrário;
  7. (endp EXP): produz t, caso a expressão EXP seja nil; produz nil, caso contrário;
  8. (consp EXP): produz t, caso a expressão EXP seja uma lista não vazia; produz nil, caso contrário;
  9. (constantp EXP): produz t, caso a expressão EXP seja uma constante; produz nil, caso contrário;
  10. (specialp EXP): produz t, caso a expressão EXP seja um símbolo especial; produz nil, caso contrário;
  11. (integerp EXP): produz t, caso a expressão EXP seja um inteiro; produz nil, caso contrário;
  12. (floatp EXP): produz t, caso a expressão EXP seja um float; produz nil, caso contrário;
  13. (rationalp EXP): produz t, caso a expressão EXP seja um número racional; produz nil, caso contrário;
  14. (complexp EXP): produz t, caso a expressão EXP seja um número complexo; produz nil, caso contrário;
  15. (stringp EXP): produz t, caso a expressão EXP seja um número string; produz nil, caso contrário;
  16. (characterp EXP): produz t, caso a expressão EXP seja um caractere; produz nil, caso contrário;
  17. (boundp SMB): produz t, caso o símbolo SMB tenha associado a si algum valor; produz nil, caso contrário;
  18. (fboundp SMB): produz t, caso o símbolo SMB tenha associado a si algum valor funcional; produz nil, caso contrário;
  19. (minusp NUM): produz t, caso o número NUM seja negativo; produz nil, caso contrário;
  20. (zerop NUM): produz t, caso o número NUM seja zero; produz nil, caso contrário;
-

21. (plusp NUM): produz t, caso o número NUM seja positivo; produz nil, caso contrário;
  22. (evenp NUM): produz t, caso o número NUM seja par; produz nil, caso contrário;
  23. (oddp NUM): produz t, caso o número NUM seja ímpar; produz nil, caso contrário;
  24. (Equal EXP<sub>1</sub> EXP<sub>2</sub>): produz t, caso a expressão EXP<sub>1</sub> seja igual à expressão EXP<sub>2</sub> (estão na mesma posição de memória); produz nil, caso contrário; só funciona com caracteres, símbolos e inteiros curtos;
  25. (eq EXP<sub>1</sub> EXP<sub>2</sub>): produz t, caso a expressão EXP<sub>1</sub> seja igual à expressão EXP<sub>2</sub>; produz nil, caso contrário; funciona com todo tipo de número, desde que tenham o mesmo tipo;
  26. (equal EXP<sub>1</sub> EXP<sub>2</sub>): produz t, caso a expressão EXP<sub>1</sub> seja igual à expressão EXP<sub>2</sub>; produz nil, caso contrário; funciona com qualquer par de expressões;
  27. (typep EXP TIP): produz t, caso a expressão EXP seja do tipo TIP; produz nil, caso contrário;
-

## Anexo I: Exercícios Propostos

### Exercícios com Números

1. Escreva uma função em LISP Puro que recebe como parâmetro um número inteiro e retorna 'T, caso ele seja negativo, ou 'NIL, caso contrário. Este é o único exercício em que fazer uma função auxiliar será aceitável.
  2. Escreva uma função em LISP Puro que recebe dois números inteiros como parâmetro. A função deverá retornar 'T, caso o primeiro número fornecido for diferente do segundo. A função deverá retornar 'NIL, caso contrário.
  3. Escreva uma função em LISP Puro que recebe dois números inteiros como parâmetro. A função deverá retornar 'T, caso o primeiro número fornecido for menor do que o segundo. A função deverá retornar 'NIL, caso contrário.
  4. Escreva uma função em LISP Puro que recebe dois números inteiros como parâmetro. A função deverá retornar 'T, caso o primeiro número fornecido for menor ou igual ao segundo. A função deverá retornar 'NIL, caso contrário.
  5. Escreva uma função em LISP Puro que recebe dois números inteiros como parâmetro. A função deverá retornar 'T, caso o primeiro número fornecido for maior do que o segundo. A função deverá retornar 'NIL, caso contrário.
  6. Escreva uma função em LISP Puro que recebe dois números inteiros como parâmetro. A função deverá retornar 'T, caso o primeiro número fornecido for maior ou igual ao segundo. A função deverá retornar 'NIL, caso contrário.
  7. Escreva uma função em LISP Puro que recebe dois números inteiros como parâmetro. A função deverá retornar a soma dos dois valores fornecidos.
-

8. Escreva uma função em LISP Puro que recebe dois números inteiros como parâmetro. A função deverá retornar a subtração dos dois valores fornecidos.
9. Escreva uma função em LISP Puro que recebe dois números inteiros como parâmetro. A função deverá retornar a multiplicação dos dois valores fornecidos.
10. Escreva uma função em LISP Puro que recebe como parâmetro um número inteiro e retorna o valor absoluto do valor fornecido.
11. Escreva uma função em LISP Puro que recebe dois números inteiros como parâmetro. A função deverá retornar a divisão inteira dos dois valores fornecidos.
12. Escreva uma função em LISP Puro que recebe dois números inteiros como parâmetro. A função deverá retornar o resto da divisão inteira dos dois valores fornecidos.
13. Escreva uma função em LISP Puro que recebe dois números inteiros, B e P, como parâmetro. A função deverá retornar  $B^P$  (B elevado a P).
14. Escreva uma função em LISP Puro que recebe como parâmetro um número natural. A função deverá retornar o fatorial do número recebido, ou 'NIL, caso o número fornecido seja negativo.
15. Escreva uma função em LISP Puro que recebe dois números naturais como parâmetro. A função deverá retornar o MDC dos dois números recebidos, ou 'NIL, caso um dos números fornecidos seja negativo.

$$\text{Saiba que } \text{mdc}(X, Y) = \begin{cases} Y, & \text{se resto}(X / Y) = 0; \\ \text{mdc}(Y, \text{resto}(X / Y)), & \text{cc.} \end{cases}$$

16. Escreva uma função em LISP Puro que recebe como parâmetro um número natural até 9 e retorna um símbolo que expressa, por extenso, o número recebido. A função deverá retornar o símbolo INVALIDO, caso o número fornecido não esteja entre 0 e 9.
  17. Escreva uma função em LISP Puro que recebe como parâmetro um número natural que expressa, em centímetros, o tamanho (L) dos lados de um quadrado. A função deverá retornar
-

um número natural que expressa, em centímetros quadrados, a área do retângulo em questão. A relação entre essas grandezas é  $\text{Area} = L^2$ .

18. Escreva uma função em LISP Puro que recebe dois parâmetros: (1) um número natural que expressa, em centímetros, o tamanho (m) do lado menor de um retângulo; e (2) um número natural que expressa, em centímetros, o tamanho (M) do lado maior do supra referido retângulo. A função deverá retornar um número natural que expressa, em centímetros quadrados, a área do retângulo em questão. A relação entre essas grandezas é  $\text{Area} = m \cdot M$
19. Escreva uma função em LISP Puro que recebe dois parâmetros: (1) um número natural que expressa, em centímetros, o tamanho da diagonal menor (d) de um losângulo; e (2) um número natural que expressa, em centímetros, o tamanho da diagonal maior (D) do supra referido losângulo. A função deverá retornar um número real que expressa, em centímetros quadrados, a área do losângulo em questão. A relação entre essas grandezas é  $\text{Area} = (d \cdot D) / 2$ . Neste exercício, use a função / para dividir, em vez de sua função de divisão.
20. Escreva uma função em LISP Puro que recebe dois parâmetros: (1) um número natural que expressa, em centímetros, o tamanho da base (B) de um paralelogramo; e (2) um número natural que expressa, em centímetros, a altura (A) do supra referido paralelogramo. A função deverá retornar um número natural que expressa, em centímetros quadrados, a área do paralelogramo em questão. A relação entre essas grandezas é  $\text{Area} = B \cdot A$ .
21. Escreva uma função em LISP Puro que recebe três parâmetros: (1) um número natural que expressa, em centímetros, o tamanho da base menor (b) de um trapézio; (2) um número natural que expressa, em centímetros, o tamanho da base maior (B) do supra referido trapézio; e (3) um número natural que expressa, em centímetros, a altura do supra referido trapézio. A função deverá retornar um número real que expressa, em centímetros quadrados, a área do trapézio em questão. A relação entre essas grandezas é  $\text{Area} = ((b+B) \cdot A) / 2$ . Neste exercício, use a função / para dividir, em vez de sua função de divisão.
22. Escreva uma função em LISP Puro que recebe três parâmetros: (1) um número natural que expressa a quantidade de lados de um polígono regular; (2) um número natural que expressa,
-

em centímetros, o tamanho da base (B) do supra referido polígono regular; e (3) um número natural que expressa, em centímetros, o tamanho da apótema (A) do supra referido polígono regular medida em centímetros de sua base (B) e de sua apótema (A), ou seja, a reta imaginária que une seu centro ao meio de sua base. A função deverá retornar um número real que expressa, em centímetros quadrados, a área do polígono regular em questão. A relação entre essas grandezas é  $Area = (Q.B.A)/2$ . Neste exercício, use a função / para dividir, em vez de sua função de divisão.

23. Escreva uma função em LISP Puro que recebe como parâmetro um número natural que expressa, em centímetros, o tamanho (R) do raio de um círculo. A função deverá retornar um número real que expressa, em centímetros quadrados, a área do círculo em questão. A relação entre essas grandezas é  $Area = \pi.R^2$ , sendo  $\pi$  constante e aproximadamente igual a 3,1415. Neste exercício, use a função / para dividir, em vez de sua função de divisão.
24. Escreva uma função em LISP Puro que recebe dois parâmetros: (1) um número natural que expressa, em gramas, o peso (P) de uma pessoa; e (2) um número natural que expressa, em centímetros, a altura (A) da pessoa em questão. A função deverá retornar um número real que expressa, o Índice de Massa Corpórea (ou BMI, Body Mass Index) da supra referida pessoa. A relação entre essas grandezas é  $BMI = P / (100.A^2)$ . Neste exercício, use a função / para dividir, em vez de sua função de divisão.
25. Escreva uma função em LISP Puro que recebe como parâmetro um número natural. A função deverá retornar a soma dos divisores do número natural dado.
26. Escreva uma função em LISP Puro que recebe três parâmetros: (1) um número natural que expressa a hora de um horário; e (2) um número natural que expressa os minutos do supra referido horário; e (3) um número natural que expressa os segundos do supra referido horário. A função deve retornar 'T, caso os parâmetros fornecidos representem um horário válido, ou 'NIL, caso contrário.
27. Escreva uma função em LISP Puro que recebe três parâmetros: (1) um número natural que expressa o dia de uma data; (2) um número natural que expressa o mês da supra referida data;
-

e (3) um número natural que expressa o ano da supra referida data. A função deverá retornar 'T, caso os parâmetros fornecidos representem uma data válida, ou 'NIL, caso contrário. Lembrar dos meses de 30 e 31 dias, bem como do mês de fevereiro e seus 28 ou 29 dias. Anos bissextos são anos divisíveis por 4 mas não por 100, ou então anos divisíveis por 400.

28. Escreva uma função em LISP Puro que recebe três números naturais como parâmetro (as medidas em centímetro de três retas). A função deverá retornar 'T, caso retas com as medidas fornecidas possam formar um triângulo, ou 'NIL, caso contrário.
29. Escreva uma função em LISP Puro que recebe três números naturais como parâmetro (as medidas em centímetro de três retas). Quando não for possível formar um triângulo com as três retas, a função deverá retornar 'NIL; quando for possível formar um triângulo com as três retas, a função deverá retornar um dos seguintes símbolos: 'EQUILATERO (no caso das três retas terem a mesma medida), 'ISOSCELES (no caso de duas delas terem a mesma medida, porém diferente da medida da terceira) ou 'ESCALENO (no caso das três retas terem medidas diferentes).
30. Escreva uma função em LISP Puro que recebe três números naturais como parâmetro (as medidas em graus de três ângulos). A função deverá retornar 'T, caso seja possível um triângulo ter tais ângulos internos, ou 'NIL, caso contrário.
31. Escreva uma função em LISP Puro que recebe três números naturais como parâmetro (as medidas em graus de três ângulos). Quando não for possível um triângulo ter tais ângulos internos, a função deverá retornar 'NIL; quando for possível um triângulo ter tais ângulos internos, a função deverá retornar um dos seguintes símbolos: 'ACUTANGULO (no caso dos três ângulos serem agudos, ou seja, menores que 90 graus), 'RETANGULO (no caso de um dos ângulos medir 90 graus) ou 'OBTUSANGULO (no caso de um dos ângulos ser obtuso, ou seja, medir mais do que 90 graus).
32. Escreva uma função em LISP Puro que recebe dois números naturais como parâmetro (os coeficientes a e b de uma equação de primeiro grau). A função deverá retornar a raiz da equação. Neste exercício, use a função / para dividir, em vez de sua função de divisão.
-

33. Escreva uma função em LISP Puro que recebe dois números inteiros como parâmetro. A função deverá retornar a soma dos números inteiros compreendidos no intervalo, incluindo os limites (no caso dos limites serem iguais, um dos limites). A função deverá retornar 'NIL, caso o primeiro número fornecido seja maior do que o segundo.
  34. Escreva uma função em LISP Puro que recebe como parâmetro um número natural. A função deverá retornar a soma dos divisores do número recebido, ou 'NIL, caso o número fornecido seja negativo.
  35. Escreva uma função em LISP Puro que recebe como parâmetro um número natural. A função deverá retornar 'T, caso o número fornecido seja primo, 'NIL, caso contrário (ou caso o número fornecido seja negativo).
  36. Escreva uma função em LISP Puro que recebe dois números naturais MIN e MAX como parâmetro. A função deverá retornar a soma dos números primos entre MIN e MAX, incluindo MIN e MAX (ou apenas um deles, caso eles sejam iguais e sejam números primos). O predicado deverá retornar 'NIL, caso MIN seja maior do que MAX ou se algum deles for negativo.
  37. Escreva uma função em LISP Puro que recebe como parâmetro um número natural. A função deverá retornar 'T, caso o número fornecido seja perfeito, ou 'NIL, caso contrário (ou caso o número fornecido seja negativo). Perfeitos são os números iguais à soma de seus divisores distintos de si próprio (6 é; 10 não é).
  38. Escreva uma função em LISP Puro que recebe dois números naturais MIN e MAX como parâmetro. A função deverá retornar a soma dos números perfeito entre MIN e MAX, incluindo MIN e MAX (ou apenas um deles, caso eles sejam iguais e sejam números perfeitos). O predicado deverá retornar 'NIL, caso MIN seja maior do que MAX ou se algum deles for negativo.
  39. Escreva uma função em LISP Puro que recebe como parâmetro dois números naturais. A função deverá retornar 'T, caso os números fornecidos sejam amigos, ou 'NIL, caso contrário (ou caso algum dos números fornecidos seja negativo). Amigos são os números que têm a
-

soma de seus divisores distintos de si próprio igual ao outro (220 e 284 são; 123 e 321 não são).

40. Escreva uma função em LISP Puro que recebe dois números naturais MIN e MAX como parâmetro. A função deverá retornar a soma da multiplicação dos pares de números amigos entre MIN e MAX, incluindo MIN e MAX (ou apenas um deles, caso eles tenham um amigo. O predicado deverá retornar 'NIL, caso MIN seja maior ou igual a MAX ou se algum deles for negativo.
41. Escreva uma função em LISP Puro que recebe como parâmetro um número natural. A função deverá retornar o número natural que tem os mesmos dígitos que o número fornecido, porém na ordem contrária (inversa).
42. Escreva uma função em LISP Puro que recebe como parâmetro um número natural. A função deverá retornar 'T, caso o número recebido seja palíndromo (mesma leitura da esquerda para a direita e da direita para a esquerda), ou 'NIL, caso contrário.

## Exercícios sobre Listas

43. Escreva uma função em LISP Puro que recebe como parâmetro uma lista e retorna a quantidade de elementos da lista dada.
  44. Escreva uma função em LISP Puro que recebe como parâmetro uma lista e retorna o primeiro elemento da lista dada.
  45. Escreva uma função em LISP Puro que recebe como parâmetro uma lista e retorna o último elemento da lista dada.
  46. Escreva uma função em LISP Puro que recebe dois parâmetros: (1) uma lista; e (2) um número de ordem (1, 2, 3, etc) ao qual chamaremos de N. A função deverá retornar o N<sup>ézimo</sup> elemento da lista dada.
-

47. Escreva uma função em LISP Puro que recebe dois parâmetros: (1) uma lista; e (2) um possível elemento da lista dada. A função deverá retornar 'T, no caso do possível elemento dado, de fato, ser elemento da lista dada. A função deverá retornar 'NIL, caso contrário.
48. Escreva uma função em LISP Puro que recebe dois parâmetros: (1) uma lista; e (2) um possível elemento da lista dada. A função deverá retornar a quantidade de vezes que o possível elemento dado ocorre como elemento da lista dada.
49. Usando a função implementada no exercício 43, escreva uma função em LISP Puro que recebe dois parâmetros: (1) uma lista; e (2) um possível elemento da lista dada. A função deverá retornar 'T, no caso do possível elemento dado, de fato, ser elemento da lista dada. A função deverá retornar 'NIL, caso contrário.
50. Sem usar a função implementada no exercício 43, escreva uma função em LISP Puro que recebe dois parâmetros: (1) uma lista; e (2) um possível elemento da lista dada. A função deverá retornar 'T, no caso do possível elemento dado, de fato, ser elemento da lista dada. A função deverá retornar 'NIL, caso contrário.
51. Escreva uma função em LISP Puro que recebe dois parâmetros: (1) uma lista; e (2) um possível elemento para a lista dada. A função deverá retornar a lista que se obtém da inclusão do possível elemento dado como primeiro elemento da lista dada.
52. Escreva uma função em LISP Puro que recebe dois parâmetros: (1) uma lista; e (2) um possível elemento para a lista dada. A função deverá retornar a lista que se obtém da inclusão do possível elemento dado como último elemento da lista dada.
53. Escreva uma função em LISP Puro que recebe três parâmetros: (1) uma lista; (2) um possível elemento para a lista dada; (3) um número de ordem (1, 2, 3, etc) ao qual chamaremos de N. A função deverá retornar a lista que se obtém da inclusão do possível elemento dado como N<sup>ézimo</sup> elemento da lista dada.
-

54. Escreva uma função em LISP Puro que recebe como parâmetro uma lista, A função deverá retornar a lista que se obtém da remoção do elemento mais à esquerda (primeiro elemento) da lista dada.
  55. Escreva uma função em LISP Puro que recebe como parâmetro uma lista, A função deverá retornar a lista que se obtém da remoção do elemento mais à direita (último elemento) da lista dada.
  56. Escreva uma função em LISP Puro que recebe dois parâmetros: (1) uma lista; e (2) um número de ordem (1, 2, 3, etc) ao qual chamaremos de N. A função deverá retornar a lista que se obtém da remoção do N<sup>º</sup> elemento da lista dada.
  57. Escreva uma função em LISP Puro que recebe dois parâmetros: (1) uma lista; e (2) um possível elemento para a lista dada. A função deverá retornar a lista que se obtém da remoção da primeira ocorrência do referido possível elemento da lista dada.
  58. Escreva uma função em LISP Puro que recebe dois parâmetros: (1) uma lista; e (2) um possível elemento para a lista dada. A função deverá retornar a lista que se obtém da remoção da última ocorrência do referido possível elemento da lista dada.
  59. Escreva uma função em LISP Puro que recebe dois parâmetros: (1) uma lista; e (2) um possível elemento para a lista dada. A função deverá retornar a lista que se obtém da remoção de todas as ocorrências do referido possível elemento da lista dada.
  60. Escreva uma função em LISP Puro que recebe duas listas como parâmetro. A função deverá retornar a lista que se obtém da concatenação das duas listas dadas.
  61. Escreva uma função em LISP Puro que recebe como parâmetro uma lista e retorna a inversão da lista dada.
-

## Exercícios sobre Listas de Números

62. Escreva uma função em LISP Puro que recebe dois parâmetros: (1) um número inteiro; e (2) uma lista de números inteiros. A função deverá retornar a lista que se obtém da inclusão ordenada do referido número inteiro na lista de números inteiros dada.
63. Usando a função implementada no exercício anterior, escreva uma função em LISP Puro que recebe como parâmetro uma lista de números inteiros. A função deverá retornar a lista que se obtém da ordenação da lista de números inteiros dada.
64. Escreva uma função em LISP Puro que recebe como parâmetro uma lista de números inteiros e retorna o menor número inteiro dela.
65. Escreva uma função em LISP Puro que recebe como parâmetro uma lista de números inteiros. A função deverá retornar a lista que se obtém da retirada da primeira ocorrência do menor número inteiro da lista de números inteiros dada.
66. Escreva uma função em LISP Puro que recebe como parâmetro uma lista de números inteiros. A função deverá retornar a lista que se obtém da retirada da última ocorrência do menor número inteiro da lista de números inteiros dada.
67. Escreva uma função em LISP Puro que recebe como parâmetro uma lista de números inteiros. A função deverá retornar a lista que se obtém da retirada de todas ocorrências do menor número inteiro da lista de números inteiros dada.
68. Escreva uma função em LISP Puro que recebe como parâmetro uma lista de números inteiros e retorna o maior número inteiro dela.
69. Escreva uma função em LISP Puro que recebe como parâmetro uma lista de números inteiros. A função deverá retornar a lista que se obtém da retirada da primeira ocorrência do maior número inteiro da lista de números inteiros dada.
-

70. Escreva uma função em LISP Puro que recebe como parâmetro uma lista de números inteiros. A função deverá retornar a lista que se obtém da retirada da última ocorrência do maior número inteiro da lista de números inteiros dada.
71. Escreva uma função em LISP Puro que recebe como parâmetro uma lista de números inteiros. A função deverá retornar a lista que se obtém da retirada de todas ocorrências do maior número inteiro da lista de números inteiros dada.
72. Escreva uma função em LISP Puro que recebe como parâmetro uma lista de números inteiros e retorna a soma dos valores presentes nela.
73. Escreva uma função em LISP Puro que recebe como parâmetro uma lista de números inteiros e retorna a média aritmética dos valores presentes nela. Neste exercício, use a função / para dividir, em vez de sua função de divisão.

## Exercícios sobre Listas Generalizadas

74. Escreva uma função em LISP Puro que recebe como parâmetro uma lista de listas de átomos. A função deverá retornar uma lista contendo as quantidades de elementos de cada uma das listas contidas na lista recebida.
75. Escreva uma função em LISP Puro que recebe como parâmetro uma lista generalizada. A função deverá retornar uma lista com os átomos contidos direta ou indiretamente na lista recebida.
76. Escreva uma função em LISP Puro que recebe dois parâmetros: (1) uma lista generalizada; e (2) um átomo. A função deverá retornar 'T, caso a lista contenha, direta ou indiretamente, o valor átomo dado. A função deverá retornar 'NIL, caso contrário.

77. Escreva uma função em LISP Puro que recebe dois parâmetros: (1) uma lista generalizada; e (2) um valor, não necessariamente atômico. A função deverá retornar 'T, caso a lista contenha, direta ou indiretamente, o valor dado. A função deverá retornar 'NIL, caso contrário.
78. Escreva uma função em LISP Puro que recebe como parâmetro uma lista generalizada. A função deverá retornar um número inteiro que expressa o MENOR nível de encaixamento presente na lista dada (1, se a lista contem ao menos um elemento atômico; 2, se a lista contem ao menos um elemento, cujos elementos sejam todos atômicos; 3, se a lista contem ao menos um elemento que seja uma lista de elementos atômicos; e assim por diante).
79. Escreva uma função em LISP Puro que recebe como parâmetro uma lista generalizada. A função deverá retornar um número inteiro que expressa o MAIOR nível de encaixamento presente na lista dada (1, se a lista contem ao menos um elemento atômico; 2, se a lista contem ao menos um elemento, cujos elementos sejam todos atômicos; 3, se a lista contem ao menos um elemento que seja uma lista de elementos atômicos; e assim por diante).
80. Escreva uma função em LISP Puro que recebe como parâmetro uma lista generalizada e retorna o átomo mais à esquerda da referida lista (ele pode estar em uma sub lista).
81. Escreva uma função em LISP Puro que recebe como parâmetro uma lista generalizada. A função deverá retornar a lista que se obtém ao retirar da lista dada seu átomo mais à esquerda (ele pode estar em uma sub lista).
82. Escreva uma função em LISP Puro que recebe como parâmetro uma lista generalizada e retorna o átomo mais à direita da referida lista (ele pode estar em uma sub lista).
83. Escreva uma função em LISP Puro que recebe como parâmetro uma lista generalizada. A função deverá retornar a lista que se obtém ao retirar da lista dada seu átomo mais à direita (ele pode estar em uma sub lista).
-

84. Escreva uma função em LISP Puro que recebe dois parâmetros: (1) um átomo; e (2) uma lista generalizada. A função deverá retornar a lista que se obtém da remoção da primeira ocorrência (seja no nível que for) do referido átomo da lista generalizada dada.
  85. Escreva uma função em LISP Puro que recebe dois parâmetros: (1) um átomo; e (2) uma lista generalizada. A função deverá retornar a lista que se obtém da remoção da última ocorrência (seja no nível que for) do referido átomo da lista generalizada dada.
  86. Escreva uma função em LISP Puro que recebe dois parâmetros: (1) um átomo; e (2) uma lista generalizada. A função deverá retornar a lista que se obtém da remoção de todas as ocorrências (seja no nível que for) do referido átomo da lista generalizada dada.
  87. Escreva uma função em LISP Puro que recebe como parâmetro uma lista generalizada de números. A função deverá retornar o maior número (seja no nível que for) da lista generalizada dada.
  88. Escreva uma função em LISP Puro que recebe como parâmetro uma lista generalizada de números. A função deverá retornar a lista que se obtém da retirada do maior número da lista generalizada dada, seja no nível que for.
  89. Escreva uma função em LISP Puro que recebe como parâmetro uma lista generalizada de números. A função deverá retornar a soma dos números (seja no nível que for) da lista de números dada.
  90. Escreva uma função em LISP Puro que recebe como parâmetro uma lista generalizada de números. A função deverá retornar a quantidade de números (seja no nível que for) da lista de números dada.
  91. Escreva uma função em LISP Puro que recebe como parâmetro uma lista generalizada de números. A função deverá retornar a média aritmética dos números (seja no nível que for) da lista de números dada. Neste exercício, use a função / para dividir, em vez de sua função de divisão.
-

## Exercícios sobre Listas de S-Expressões

Em todos os exercícios abaixo, toda menção a disciplinas deve ser entendida como fazendo referência a uma S-Expressão que representa os resultados acadêmicos de uma disciplina e que são assim constituídos: no CAR, encontraremos o nome da disciplina, no CADR, encontraremos a nota obtida na disciplina e no CDDR, encontraremos a frequência obtida na disciplina.

92. Escreva uma função em LISP Puro que recebe como parâmetro uma lista de disciplinas. A função deverá retornar o nome da disciplina com maior nota da lista de disciplinas dada (suponha que tal disciplina exista e que ela seja única).
  93. Escreva uma função em LISP que recebe como parâmetro uma lista de disciplinas. A função deverá retornar a soma das notas da lista de disciplinas dada.
  94. Escreva uma função em LISP que recebe como parâmetro uma disciplina e uma lista de disciplinas ordenada em ordem crescente de nota. A função deverá retornar a lista que se obtem da inclusão ordenada do referido functor na lista de functors dada.
  95. Escreva uma função em LISP que recebe como parâmetro uma lista de disciplinas. A função deverá retornar a lista que se obtem da ordenação, em ordem crescente de nota, da lista de functors dada.
  96. Escreva uma função em LISP que recebe como parâmetro uma lista de disciplinas. A função deverá retornar a lista que se obtem da ordenação, em ordem decrescente de frequência, da lista de functors dada.
  97. Escreva uma função em LISP que recebe como parâmetro uma lista de disciplinas. A função deverá retornar o nome da matéria com menor frequência da lista de functors dada (suponha que tal disciplina exista e que ela seja única).
-

98. Escreva uma função em LISP que recebe como parâmetro uma lista de disciplinas. A função deverá retornar a lista que se obtém da retirada da disciplina de maior nota da lista de functors dada (suponha que tal disciplina exista e que ela seja única).
99. Escreva uma função em LISP que recebe como parâmetro uma lista de disciplinas. A função deverá retornar a lista que se obtém da retirada da disciplina de menor frequência da lista de functors dada (suponha que tal disciplina exista e que ela seja única).
100. Escreva uma função em LISP que recebe como parâmetro uma lista de disciplinas. A função deverá retornar o número real que representa a média aritmética das notas da lista de functors dada.
-

## Anexo II: Referências

[Sebesta89] Sebesta, R.W., “Concepts of Programming Languages”, The Benjamin/Cummings Publishing Company, Inc., 1989.

[Baranauskas93] Baranauskas, M.C.C., “Procedimento, Função, Objeto ou Lógica? Linguagens de Programação vistas pelos seus Paradigmas” *in* Computadores e Conhecimento - Repensando a Educação, Valente, J.A. (Org.), NIED, UNICAMP, 1993.

Prof. André Reis  
dos Reis  
de Carvalho  
Gomes